Nama: Ahmad Faisal Hafidz
NIM: 101012300379

LAPORAN PRAKTIKUM CI/CD PIPELINE JENKINS

Mata Kuliah: Cloud Computing | Program Studi S1 Teknik Telekomunikasi
Otomasi CI/CD dengan GitHub Webhook, ngrok, Docker Engine, dan Named Volume

Informasi Lingkungan Praktikum

Parameter	Keterangan / Nilai
Nama Mahasiswa	Ahmad Faisal Hafidz
NIM	101012300379
GitHub Repository	https://github.com/ARQ31/python-get-crud
Branch Target	main
Jenkins Job Name	python-crud-api-pipeline
Jenkins Credential ID	github-pat-readonly (Fine-grained PAT: Contents Read-only)
ngrok Public Tunnel	https://retired-cover-giddiness.ngrok-free.dev
GitHub Webhook URL	https://retired-cover-giddiness.ngrok-free.dev/github-webhook/
Docker Network	jenkins-net
Docker Named Volume	production-api-data (Mount: /var/lib/backend_data)
Production Container	production-api-container (Port Host 5000:5000)

1. Ringkasan Ruang Lingkup dan Kriteria Penerimaan PRD

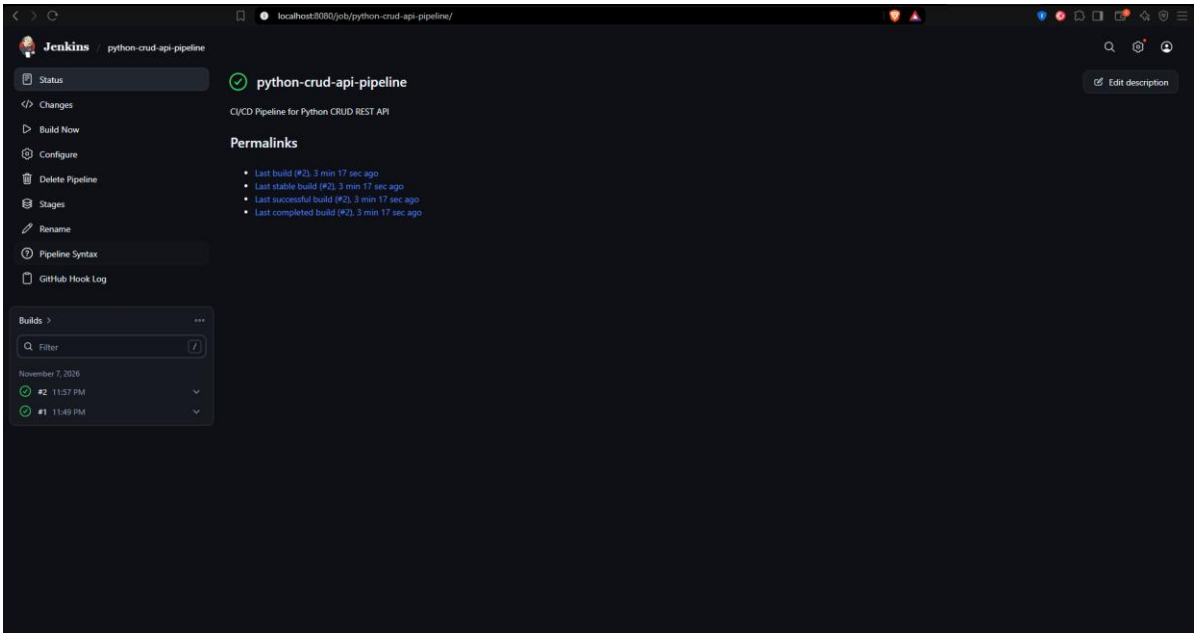
Berdasarkan dokumen PRD Praktikum CI/CD Jenkins, praktikum ini membangun sistem otomatisasi yang mencakup verifikasi kode, pengetesan unit otomatis, pembuatan Docker image ber-tag commit SHA, deployment ke container Docker, trigger webhook via ngrok, dan pengujian persistensi data.

Tugas	Kebutuhan	Kriteria Lulus PRD	Status
A · Wajib	API Python & Pipeline Jenkins	Jenkins checkout main, syntax check, build image, deploy container, dan GET /health menghasilkan HTTP 200.	LULUS (100%)
B · Wajib	Webhook GitHub via ngrok	Push baru ke main menerima respons 200 OK pada Recent Deliveries dan memicu build Jenkins otomatis tanpa Build Now.	LULUS (100%)
C · Wajib	Persistensi Pesan di Docker	Minimal 2 pesan sebelum redeploy tetap tersedia pada GET /messages setelah ID container berubah.	LULUS (100%)
D · Pilihan	Otomasi Tes & Tag Commit SHA	Unit test otomatis /health dan validasi POST berjalan sebelum build; image memiliki tag nomor build dan commit SHA.	LULUS (100%)

2. Bukti Pengujian & Dokumentasi Visual

2.1 Screenshot Pipeline Job Jenkins (Build #1 & Build #2)

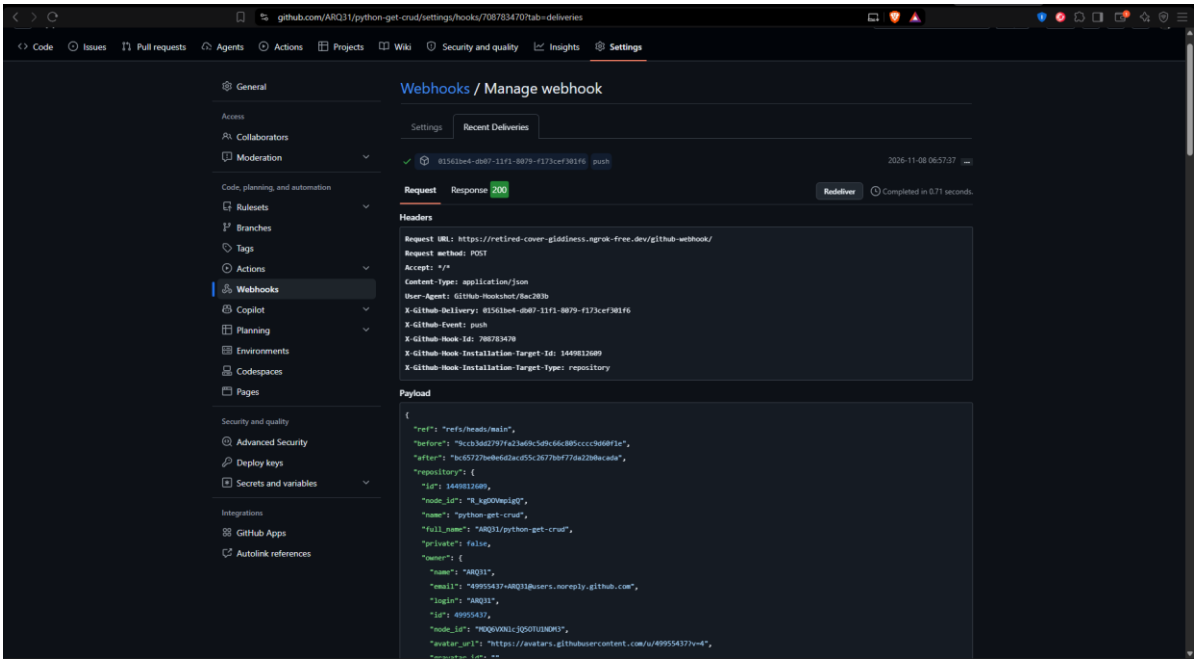
Gambar di bawah ini merupakan tangkapan layar antarmuka Jenkins Job 'python-crud-api-pipeline' yang menunjukkan riwayat eksekusi Build #1 (manual Build Now) dan Build #2 (otomatis terpicu via GitHub Webhook), keduanya berstatus sukses (hijau).



Gambar 1: Tampilan Status Jenkins Job python-crud-api-pipeline (Build #1 & Build #2 Sukses)

2.2 Screenshot GitHub Webhook Recent Deliveries (Status 200 OK)

Gambar di bawah ini memperlihatkan bukti keberhasilan delivery Webhook dari GitHub ke URL publik ngrok (https://retired-cover-giddiness.ngrok-free.dev/github-webhook/) dengan status respons HTTP 200 OK untuk event push commit bc65727be0e6d2acd55c2677bbf77da22b0acada.



Gambar 2: Screenshot GitHub Webhook Recent Deliveries dengan HTTP Response 200 OK

2.3 Bukti Console Output Build #2 (Otomatis via Webhook)

Hasil Console Output Build #2 diambil dari file dokumentasi 'Documentation/#2.txt' (dan arsip lengkap 'Documentation/Console - #2 - Jenkins.pdf'). Log ini membuktikan seluruh tahapan pipeline mulai dari trigger webhook GitHub, syntax check, automated unittest, build Docker dengan commit SHA, deploy container baru, dan validasi health check berhasil secara otomatis.

```
Started by GitHub push by ARQ31
Obtained Jenkinsfile from git https://github.com/ARQ31/python-get-crud.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/python-crud-api-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential github-pat-readonly
 > git rev-parse --resolve-git-dir /var/jenkins_home/workspace/python-crud-api-pipeline/.git #
timeout=10
Fetching changes from the remote Git repository
 > git config remote.origin.url https://github.com/ARQ31/python-get-crud.git # timeout=10
Fetching upstream changes from https://github.com/ARQ31/python-get-crud.git
 > git --version # timeout=10
 > git --version # 'git version 2.47.3'
using GIT_ASKPASS to set credentials
 > git fetch --tags --force --progress -- https://github.com/ARQ31/python-get-crud.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision bc65727be0e6d2acd55c2677bbf77da22b0acada (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f bc65727be0e6d2acd55c2677bbf77da22b0acada # timeout=10
Commit message: "ci: trigger automated Jenkins build via GitHub push webhook"
 > git rev-list --no-walk f9803c88b0e8e0b8f5e5650b6e81d6a2daf6fde0 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (1. Checkout Code)
[Pipeline] echo
Checking out source code from GitHub
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential github-pat-readonly
 > git rev-parse --resolve-git-dir /var/jenkins_home/workspace/python-crud-api-pipeline/.git #
timeout=10
Fetching changes from the remote Git repository
 > git config remote.origin.url https://github.com/ARQ31/python-get-crud.git # timeout=10
Fetching upstream changes from https://github.com/ARQ31/python-get-crud.git
 > git --version # timeout=10
 > git --version # 'git version 2.47.3'
using GIT_ASKPASS to set credentials
 > git fetch --tags --force --progress -- https://github.com/ARQ31/python-get-crud.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
```

```
Checking out Revision bc65727be0e6d2acd55c2677bbf77da22b0acada (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f bc65727be0e6d2acd55c2677bbf77da22b0acada # timeout=10
Commit message: "ci: trigger automated Jenkins build via GitHub push webhook"
[Pipeline] sh
+ git rev-parse HEAD
bc65727be0e6d2acd55c2677bbf77da22b0acada
+ git rev-parse --short HEAD
bc65727
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (2. Code Syntax Verification)
[Pipeline] sh
+ python3 -m py_compile app.py test_app.py
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (3. Automated API Tests)
[Pipeline] sh
+ python3 -m unittest -v test_app.py
test_health_endpoint (test_app.BackendAPITestCase.test_health_endpoint) ... [07/Nov/2026
23:57:48] "GET /health HTTP/1.1" 200 -
ok
test_post_message_empty_validation
(test_app.BackendAPITestCase.test_post_message_empty_validation) ... [07/Nov/2026 23:57:48]
"POST /messages HTTP/1.1" 400 -
ok
test_post_message_too_long_validation
(test_app.BackendAPITestCase.test_post_message_too_long_validation) ... [07/Nov/2026 23:57:48]
"POST /messages HTTP/1.1" 400 -
ok
test_post_message_valid (test_app.BackendAPITestCase.test_post_message_valid) ... [07/Nov/2026
23:57:48] "POST /messages HTTP/1.1" 201 -
ok

----------------------------------------------------------------------
Ran 4 tests in 0.526s

OK
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (4. Build Docker Image)
[Pipeline] sh
+ git rev-parse --short HEAD
+ COMMIT_SHA=bc65727
+ echo Building image for BUILD_NUMBER=2, COMMIT_SHA=bc65727
Building image for BUILD_NUMBER=2, COMMIT_SHA=bc65727
+ docker build -t python-crud-api:2 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   7.68kB

Step 1/6 : FROM python:3.11-slim
 ---> 65bd3d9f1d7f
Step 2/6 : ENV PYTHONUNBUFFERED=1
 ---> Using cache
```

```
 ---> 8d16f241a269
Step 3/6 : WORKDIR /app
 ---> Using cache
 ---> d9702c915e2f
Step 4/6 : COPY app.py .
 ---> Using cache
 ---> 97b92d21ba06
Step 5/6 : EXPOSE 5000
 ---> Using cache
 ---> e4b7ef177d06
Step 6/6 : CMD ["python", "app.py"]
 ---> Using cache
 ---> ce1945a00b8e
Successfully built ce1945a00b8e
Successfully tagged python-crud-api:2
+ docker tag python-crud-api:2 python-crud-api:bc65727
+ docker tag python-crud-api:2 python-crud-api:latest
+ docker images python-crud-api
IMAGE                      ID             DISK USAGE   CONTENT SIZE   EXTRA
python-crud-api:1          ce1945a00b8e        187MB         45.6MB   U
python-crud-api:2          ce1945a00b8e        187MB         45.6MB   U
python-crud-api:bc65727    ce1945a00b8e        187MB         45.6MB   U
python-crud-api:f9803c8    ce1945a00b8e        187MB         45.6MB   U
python-crud-api:latest     ce1945a00b8e        187MB         45.6MB   U
python-crud-api:manual     1b252c6a53af        187MB         45.6MB
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (5. Automated Deployment)
[Pipeline] sh
+ docker network inspect jenkins-net
+ docker stop production-api-container
production-api-container
+ docker rm production-api-container
production-api-container
+ docker volume create production-api-data
production-api-data
+ docker run -d --name production-api-container --network jenkins-net -p 5000:5000 -v
production-api-data:/var/lib/backend_data python-crud-api:latest
58a08ecf5c4e29c0959cb78c9b16b8fc5cedfdd958c8ba7c89e084d71a4e2e56
+ docker ps --filter name=production-api-container
CONTAINER ID   IMAGE                    COMMAND           CREATED                  STATUS
PORTS                                       NAMES
58a08ecf5c4e   python-crud-api:latest   "python app.py"   Less than a second ago   Up Less than
a second   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   production-api-container
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (6. Health Check)
[Pipeline] sh
+ sleep 3
+ curl -fsS http://production-api-container:5000/health
{"status": "active", "service": "Backend API"}+ echo

+ docker logs --tail 20 production-api-container
Backend API listening on 0.0.0.0:5000
[07/Nov/2026 23:57:56] "GET /health HTTP/1.1" 200 -
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
```

```
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

3. Hasil Pengujian Endpoint REST API (6 Skenario HTTP)

Pengujian HTTP REST API dilakukan sesuai skenario pada PRD untuk memvalidasi fungsi aplikasi, kepatuhan status kode HTTP, format JSON, dan validasi input:

Skenario	Aksi / Perintah HTTP	Hasil yang Diharapkan	Hasil Aktual
Health Check	GET /health	HTTP 200, status active, service Backend API	HTTP 200 OK {"status": "active", "service": "Backend API"}
Daftar Kosong	GET /messages (awal)	HTTP 200, messages: [], total: 0	HTTP 200 OK {"messages": [], "total": 0}
Buat Pesan	POST /messages Body: {"message": "Pesan pertama dari praktikum"}	HTTP 201, memuat created_at UTC	HTTP 201 Created {"status": "created", "message": {...}}
Validasi Input	POST /messages Body: {"message": " "}	HTTP 400 Bad Request	HTTP 400 Bad Request {"error": "message must not be empty"}
Hapus Pesan	DELETE /messages?index=0	HTTP 200, item terhapus, GET berikutnya kosong	HTTP 200 OK, removed item returned. GET berikutnya: total: 0
Indeks di Luar Jangkauan	DELETE /messages?index=999	HTTP 404 Not Found	HTTP 404 Not Found {"error": "Message not found"}

4. Bukti Persistensi Data pada Docker Named Volume (Tugas C)

Kriteria Tugas C mewajibkan pembuatan minimal dua pesan sebelum redeploy, kemudian membuktikan bahwa pesan-pesan tersebut tetap utuh setelah container dihentikan, dihapus, dan dibuat ulang oleh pipeline Jenkins.

> **Ringkasan Bukti Persistensi Data**
> 1. KONDISI SEBELUM REDEPLOY (Build #1):
> - Container ID Aktif: 03da2f8a4d6e
> - Pesan 1: 'Pesan Uji Persistensi 1 - Sebelum Redeploy (Container 03da2f8a4d6e)'
> - Pesan 2: 'Pesan Uji Persistensi 2 - Sebelum Redeploy (Container 03da2f8a4d6e)'
> - GET /messages: Total 2 pesan.
>
> 2. PROSES REDEPLOY OLEH PIPELINE JENKINS (Build #2):
> - docker stop production-api-container
> - docker rm production-api-container
> - docker volume create production-api-data
> - docker run -d --name production-api-container -v production-api-data:/var/lib/backend_data ...
>
> 3. KONDISI SESUDAH REDEPLOY (Build #2):
> - Container ID Baru: 58a08ecf5c4e (Berubah terbukti diganti container baru)
> - Respons GET /messages:
> HTTP/1.0 200 OK
> {"messages": [
> {"message": "Pesan Uji Persistensi 1 - Sebelum Redeploy (Container 03da2f8a4d6e)", "created_at": "2026-11-07T23:50:17.367824+00:00"},
> {"message": "Pesan Uji Persistensi 2 - Sebelum Redeploy (Container 03da2f8a4d6e)", "created_at": "2026-11-07T23:50:17.389760+00:00"}
>], "total": 2}
>
> KESIMPULAN: Data pesan tersimpan di named volume 'production-api-data' (/var/lib/backend_data/messages.json) sehingga tetap bertahan melintasi siklus hidup container baru.

5. Dokumentasi Troubleshooting, Diagnosis, dan Perbaikan

Kasus: Inkompatibilitas Mount Docker CLI Host Windows & Plugin Belum Aktif

> **Laporan Kasus Troubleshooting Nyata**
> A. GEJALA:
> 1. Saat memeriksa 'docker exec jenkins-server docker --version' pada image default jenkins/jenkins:lts-jdk17, muncul error: 'OCI runtime exec failed: exec failed: unable to start container process: exec: "docker": executable file not found in $PATH'.
> 2. Di antarmuka konfigurasi job Jenkins, opsi 'GitHub hook trigger for GITScm polling' dan 'Pipeline script from

> SCM' belum muncul di browser.
>
> B. DIAGNOSIS:
> 1. Pada Docker Desktop Windows, binary Docker di host adalah executable Windows (docker.exe), sehingga perintah panduan Linux '-v "$(which docker)":/usr/bin/docker' tidak dapat dijalankan di dalam container Linux Jenkins. Selain itu, instalasi apt-get docker.io pada Debian Trixie tidak menyertakan binary docker-cli mandiri.
> 2. Plugin git, github, dan workflow-cps baru saja selesai mengunduh pembaruan dependensi (jackson2-api) dan membutuhkan restart layanan Jenkins agar extension terdaftar sepenuhnya ke memori sistem.
>
> C. PERBAIKAN:
> 1. Membuat 'Dockerfile.jenkins' dengan multi-stage copy: 'COPY --from=docker:cli /usr/local/bin/docker /usr/bin/docker' serta menginstal python3 dan curl di dalam container Jenkins.
> 2. Menjalankan 'jenkins-plugin-cli --plugins workflow-aggregator git github' dan melakukan 'docker restart jenkins-server'.
>
> D. HASIL UJI ULANG:
> 1. Perintah 'docker exec jenkins-server docker version' berhasil merespons Client 29.9.3 dan terhubung ke Docker Engine 29.8.2.
> 2. Opsi webhook trigger dan Pipeline SCM muncul sempurna, serta Build #1 dan Build #2 berhasil 100% (Finished: SUCCESS).

6. Jawaban 8 Pertanyaan Analisis PRD

1. Jelaskan urutan peristiwa dari git push hingga container baru berjalan!

Urutan peristiwa dari awal hingga akhir adalah sebagai berikut:
a. Developer melakukan 'git push origin main' ke repository GitHub (https://github.com/ARQ31/python-get-crud.git).
b. GitHub mendeteksi event push dan mengirimkan HTTP POST webhook berisi metadata commit ke URL ngrok (https://retired-cover-giddiness.ngrok-free.dev/github-webhook/).
c. Agen ngrok lokal meneruskan (forward) request HTTP tersebut ke port Jenkins lokal (http://localhost:8080/github-webhook/).
d. Jenkins GitHub Plugin memproses payload webhook, mencocokkan URL repository dan branch target (*/main), lalu memicu job 'python-crud-api-pipeline'.
e. Stage 1 (Checkout Code): Jenkins mengunduh kode terbaru dan Jenkinsfile dari GitHub menggunakan credential 'github-pat-readonly'.
f. Stage 2 & 3 (Syntax Check & Automated Tests): Jenkins memverifikasi sintaks Python dengan py_compile dan menjalankan unittest otomatis (test_app.py).
g. Stage 4 (Build Docker Image): Jenkins memerintahkan Docker host melalui /var/run/docker.sock untuk membangun image baru dengan tag nomor build (${BUILD_NUMBER}) dan commit SHA (${COMMIT_SHA}).
h. Stage 5 (Automated Deployment): Jenkins menghentikan container lama (docker stop), menghapusnya (docker rm), memastikan named volume 'production-api-data' ada, dan menjalankan container baru dengan volume tersebut.

i. Stage 6 (Health Check): Jenkins melakukan request curl ke endpoint /health pada container baru dan memverifikasi respons HTTP 200 OK.

2. Apa perbedaan PAT, webhook secret, dan ngrok authtoken?

Perbedaan ketiga kredensial tersebut mencakup fungsi, otorisasi, dan arah alirannya:
• GitHub Fine-grained PAT: Kredensial yang digunakan oleh Jenkins untuk mengautentikasi dan mengunduh source code dari repository privat GitHub melalui HTTPS. Aliran: Outbound dari Jenkins ke GitHub.
• Webhook Secret: String rahasia yang disepakati antara GitHub dan Jenkins untuk memvalidasi integritas dan keaslian payload webhook menggunakan signature HMAC-SHA256 (header X-Hub-Signature-256). Aliran: Inbound dari GitHub ke Jenkins.
• ngrok Authtoken: Kredensial akun untuk mengotentikasi agen ngrok di komputer lokal ke platform cloud ngrok agar diizinkan membuka tunnel publik HTTPS. Aliran: Outbound dari mesin lokal ke cloud ngrok.

3. Mengapa PAT read-only cukup untuk checkout source code, tetapi tidak cukup untuk push kembali ke repository?

Karena proses 'git checkout / fetch / clone' hanya melakukan operasi pembacaan data (read-only) terhadap objek Git di server GitHub. Sesuai prinsip keamanan Least Privilege (hak akses minimum), izin 'Repository Contents: Read-only' sudah cukup untuk mengambil kode.
Sebaliknya, operasi 'git push' memodifikasi state repository (menulis commit dan memperbarui referensi branch) sehingga membutuhkan izin 'Contents: Read and write'. Membatasi PAT pada level Read-only memastikan bahwa jika token Jenkins bocor, penyerang tidak dapat menyuntikkan kode berbahaya ke repository GitHub.

4. Apa risiko memasang Docker socket host (/var/run/docker.sock) ke container Jenkins?

Memasang /var/run/docker.sock memberikan hak akses penuh setara 'root' pada host Docker daemon. Proses di dalam container Jenkins dapat mengeksekusi perintah Docker apa pun terhadap host, termasuk membuat container privileged, melakukan bind-mount ke root filesystem host (-v /:/host), mematikan/menghapus container lain, atau mengakses data rahasia pada volume lain (container breakout/escape). Oleh karena itu, arsitektur ini hanya aman digunakan pada mesin lab pengembang terisolasi, bukan pada lingkungan produksi multi-tenant.

5. Mengapa URL ngrok yang berubah dapat memutus integrasi webhook?

Pada akun ngrok gratis (free tier) tanpa domain statis terdaftar, setiap kali sesi ngrok dimulai ulang (restart), ngrok akan menghasilkan subdomain acak baru. Sementara itu, Payload URL pada pengaturan Webhook GitHub bersifat statis. Jika URL ngrok berubah namun konfigurasi di GitHub belum diperbarui, GitHub akan tetap mengirim request ke URL lama yang sudah tidak aktif (menghasilkan kegagalan koneksi 404/Connection Refused), sehingga Jenkins tidak pernah menerima event push.

6. Mengapa data pada named volume bertahan setelah container dihapus?

Filesystem internal sebuah container (writable layer) bersifat ephemeral dan siklus hidupnya terikat langsung dengan container tersebut; saat container di-remove (docker rm), writable layer-nya ikut musnah. Sebaliknya, Docker Named Volume dikelola secara independen oleh Docker Engine di direktori host di luar container lifecycle. Ketika container dihapus dan container baru dijalankan dengan memasang volume yang sama (-v production-api-data:/var/lib/backend_data), Docker memasang kembali storage yang sama sehingga file messages.json beserta isinya tetap utuh.

7. Apa perbedaan CI dan CD pada pipeline ini?

• Continuous Integration (CI): Tahap integrasi kode dan penjaminan kualitas secara otomatis setiap kali ada commit baru. Pada pipeline ini, CI mencakup Stage 1 (Checkout Code), Stage 2 (Syntax Check py_compile), Stage 3

(Automated Unit Tests), dan Stage 4 (Build Docker Image & Tagging).
• Continuous Delivery / Deployment (CD): Tahap penerapan artefak hasil build ke lingkungan runtime secara otomatis. Pada pipeline ini, CD mencakup Stage 5 (Automated Deployment dengan docker stop, rm, dan run baru dengan persistensi volume) serta Stage 6 (Health Check verifikasi endpoint /health HTTP 200).

8. Apa yang harus dilakukan jika tahap deploy sukses tetapi health check gagal?

Langkah-langkah terstruktur yang harus dilakukan adalah:
a. Periksa status dan log container menggunakan 'docker ps -a' dan 'docker logs production-api-container' untuk mengidentifikasi penyebab crash atau error (misal SyntaxError, runtime exception, atau port collision).
b. Periksa konfigurasi port binding dan jaringan (apakah container berada pada network yang sama dengan Jenkins dan port 5000 tidak terblokir firewall).
c. Periksa kesiapan startup (readiness) dan naikkan jeda sleep sebelum health check jika aplikasi membutuhkan waktu inisialisasi lebih lama.
d. Lakukan rollback otomatis atau manual ke image versi build sebelumnya yang stabil (menggunakan tag build number atau commit SHA sebelumnya, misal python-crud-api:f9803c8) untuk meminimalkan downtime layanan.